DAA UNIT 2

Q3(a) Explain P, NP, NP-Hard and NP-Complete problems with example. [7 Marks]

1. P Problems

P (Polynomial time) problems are problems that can be **solved in polynomial time** by a deterministic algorithm.

Examples: Binary Search, Sorting, Shortest Path

Example: Merge Sort → O(nlogn)

2. NP Problems

NP (Nondeterministic Polynomial time) problems are problems whose solutions can be **verified in polynomial time**.

In simple words, if someone gives us a solution, we can check whether it is correct efficiently.

Example: Sudoku, Hamiltonian Cycle.

3. NP-Hard Problems

A problem is **NP-Hard** if **every problem in NP can be reduced to it in polynomial time**.

An NP-Hard problem **does not necessarily belong to NP**.

Example: Travelling Salesman Problem (optimization version).

4. NP-Complete Problems

A problem is **NP-Complete** if:

1. It belongs to **NP**, and
2. It is **NP-Hard**.

Therefore: NP-Complete=NP+NP-Hard

Examples:

- 0/1 Knapsack (decision version)
- Hamiltonian Cycle
- SAT
- Travelling Salesman Problem (decision version)

Relationship P ⊆ NP

And: NP-Complete ⊆ NP

NP-Hard problems may lie **outside NP**.

Q3(b) Best, Average and Worst Case Analysis. Analyze the given algorithm. [8]

Case Analysis of Algorithms

1. Best Case - The **best case** is the input for which the algorithm takes the **minimum number of operations**.

2. Average Case - The **average case** is the expected running time for **typical/all possible inputs**.

3. Worst Case - The **worst case** is the input for which the algorithm takes the **maximum number of operations**.

Given Algorithm - The given algorithm is essentially **Insertion Sort**.

```
for i = 0 to n-1
   j = i-1
   key = a[i]
   while (j >= 0 && a[j] > key)
      a[j+1] = key
      j--
```

Best Case - The best case occurs when the array is **already sorted**.

Example: [1, 2, 3, 4, 5]

For every i, the condition: a[j] > key

is false immediately.

Therefore, the inner loop executes approximately once per outer iteration. $T(n)=O(n)$

Best Case = O(n)

Worst Case - The worst case occurs when the array is in **reverse sorted order**.

Example: [5, 4, 3, 2, 1]

For every new element, all previous elements have to be shifted.

Number of comparisons/shifts: $1+2+3+\cdots+(n-1) = n(n-1) / 2$

Therefore: $T(n) = O(n^2)$

Worst Case = $O(n^2)$

Average Case - In the average case, approximately **half of the previous elements** are shifted for each element.

Therefore, the total operations are proportional to: $(1 / 2) * (1+2+\cdots+(n-1)) = n(n-1) / 4$

Ignoring constants and lower-order terms: $T(n) = O(n^2)$

Average Case = $O(n^2)$

Space Complexity: $O(1)$ — insertion sort is an in-place algorithm.

Q4(a) What is NP-Complete class? Prove Vertex Cover is NP-Complete. [8]

NP-Complete Problems

A problem is called **NP-Complete** if:

1. It belongs to **NP**, and
2. It is **NP-Hard**.

Thus, NP-Complete = NP + NP-Hard

Examples: **SAT, 3-SAT, Vertex Cover, Hamiltonian Cycle, Clique**.

Vertex Cover Problem

Given a graph $G = (V, E)$ and an integer k, the **Vertex Cover** problem asks:

Is there a set $C \subseteq V$, with $|C| <= k$, such that every edge in E has at least one endpoint in C?

Proving Vertex Cover is NP-Complete

We prove two things:

1. Vertex Cover belongs to NP

Suppose a set C is given as a solution.

We can check in polynomial time whether:

- $|C| <= k$
- Every edge has at least one endpoint in C.

Therefore, Vertex Cover belongs to **NP**. Vertex Cover $\in$ NP

2. Vertex Cover is NP-Hard

We can reduce a known NP-Complete problem, **Clique**, to Vertex Cover in polynomial time.

For a graph (G), a set of vertices (S) forms a clique of size (k) in (G) **iff** the complementary set V−S

forms a vertex cover of size: |V|-k
Thus, Clique ≤p Vertex Cover

Since **Clique is NP-Complete**, Vertex Cover is **NP-Hard**.

Conclusion - Since Vertex Cover is: in **NP**, and **NP-Hard**,

Therefore, Vertex Cover is NP-Complete.

Q4(b) Explain 3-SAT problem with example.[7]

3-SAT (3-Satisfiability) is a special case of the Boolean SAT problem.

A 3-SAT formula is in **CNF (Conjunctive Normal Form)** where each clause contains **exactly three literals**.

The problem asks: Is there an assignment of TRUE/FALSE values to variables such that the entire Boolean formula becomes TRUE?

Example - Consider: F = (x1 ∨ x2 ∨ ¬x3) ∧ (¬x1 ∨ x3 ∨ x4)

There are two clauses, and each clause contains three literals.

Take: x1 = TRUE, x2 = FALSE, x3 = FALSE, x4 = TRUE

First clause: (TRUE ∨ FALSE ∨ TRUE) = TRUE

Second clause: (FALSE ∨ FALSE ∨ TRUE) = TRUE

Therefore: F = TRUE ∧ TRUE = TRUE

Hence, the formula is **satisfiable**.

Important Points :-

- 3-SAT is a **decision problem**.
- Each clause contains **3 literals**.
- It is an **NP-Complete problem**.
- 3-SAT is widely used in proving other problems to be NP-Complete through polynomial-time reductions.

```
void select (int A[ ])
 {// A is an array of n numbers
  for (i=1;i<n-1; i++)
  {
     min_pos=i;
     min_val=A[i];
     for(j=i+1; j<=n;j++)
     {
        if(A[j] <min_val) then
        {
          min_pos=j;
          min_val=A[j];
        }
     }
    A[min_post]=A[i];
    A[i]=min_val;
  }
 }
```


Q3(a) Analyze the complexity of Selection Sort. [6 Marks]

The given algorithm is **Selection Sort**.

Outer loop

The outer loop runs: n – 1 times.

Inner loop

For each value of i, the inner loop compares the remaining

elements.

Number of comparisons: (n−1)+(n−2)+⋯+1 = n(n - 1) / 2

Therefore, T(n) = n(n−1) / 2

Ignoring constants and lower-order terms: T(n) = $O(n^2)$

Best Case

Even if the array is already sorted, the algorithm still searches for the minimum in the remaining portion. Best Case = $O(n^2)$

Average Case

For a randomly ordered array, the same number of comparisons are performed. Average Case = $O(n^2)$

Worst Case

Even in the worst arrangement, the inner loop performs the same number of comparisons. Worst Case = $O(n^2)$

Space Complexity

Only a few extra variables such as min_pos, min_val, i, and j are used. Space = $O(1)$

Assumptions

- Array contains n elements.
- Comparison and assignment take constant time, i.e. $O(1)$
- No extra array is used.

Key point: Selection Sort always performs approximately n(n – 1) / 2 comparisons, regardless of the initial arrangement of elements.

Q3(b) If an algorithm has a time complexity of both $O(f(n))$ and $\Omega(g(n))$ for the same function f(n) and g(n), then what does it imply? Explain. [3 marks]

If an algorithm has both: $T(n)=O(f(n))$ and $T(n)=\Omega(g(n))$

for the same function, i.e. $f(n) = g(n)$, then: $T(n)=\Theta(f(n))$

Explanation

- $O(f(n))$ gives the upper bound.
- $\Omega(f(n))$ gives the lower bound.
- When both bounds are the same, the function has a tight bound.

Example -

If: $T(n)=O(n^2)$ and $T(n) = \Omega(n^2)$ then: $T(n)=\Theta(n^2)$

Q3(c) What do Ω and Θ notations mean? When do we use O notation? [6 marks]

1. Big Omega Ω

Ω notation represents the **lower bound** of an algorithm's running time.

It tells us that the algorithm takes **at least** a certain amount of time for large n.

Example: $T(n)=\Omega(n)$ means the algorithm requires at least linear time.

Used for: Lower-bound analysis.

2. Big Theta Θ

Θ notation represents the **tight bound** of an algorithm.

It gives both the upper and lower bounds.

If: $T(n)=O(f(n))$ and $T(n)=\Omega(f(n))$

then: $T(n)=\Theta(f(n))$

Example: $T(n)=\Theta(n^2)$

means the running time grows exactly at the order of n^2.

3. Big O

Big O notation represents the **upper bound** of an algorithm's running time.

It tells us the maximum growth rate of the algorithm for large input sizes.

Example: $T(n)=O(n^2)$

means the running time does not grow faster than the order of n^2.

When do we use O notation?

We mainly use **Big O** to express the **worst-case time or space complexity** of an algorithm.

For example, Linear Search has: O(n)

worst-case time complexity.

Easy way to remember: O → Upper, Ω → Lower, Θ → Exact/Tight order.

Q4(a) What is Polynomial Time Reducibility? What is its importance? [6 Marks]

Polynomial Time Reducibility

A problem A is **polynomial-time reducible** to problem B, written as: $A \leq_p B$

if every instance of problem A can be **converted into an instance of B in polynomial time**, such that solving B also solves A.

In simple words: If we can transform problem A into problem B efficiently, then A is polynomially reducible to B.

Example

If: $A \leq_p B$ and we already know that B can be solved efficiently, then A can also be solved efficiently.

Importance

1. **Used to compare problem difficulty.**
2. **Used to prove that a problem is NP-Hard or NP-Complete.**
3. Helps identify whether two problems have similar computational difficulty.
4. Provides a systematic way to transform one problem into another.
5. If an NP-Complete problem can be reduced to a problem B, then B is at least as hard as that NP-Complete problem.

Q4(b) What do understand by NP complete and NP hard problems? Give examples. [6 Marks]

NP-Complete - NP-Complete = NP + NP-Hard

A problem is **NP-Complete** if:

1. It belongs to **NP**, and
2. It is **NP-Hard**.

Examples: 3-SAT, Vertex Cover, Hamiltonian Cycle, Clique

NP-Hard

A problem is **NP-Hard** if **every problem in NP can be reduced to it in polynomial time**. An NP-Hard problem **need not belong to NP**.

Examples: Travelling Salesman Problem (optimization version), Halting Problem

Difference

NP-Complete	**NP-Hard**
Must be in NP	Need not be in NP
Must be NP-Hard	At least as hard as NP problems
Solutions can be verified in polynomial time	Verification need not be polynomial
Example: 3-SAT	Example: TSP optimization

Q4(c) Is $6n^3=\Theta(n^2)$? Justify. [3 Marks]

No.

For: $6n^3=\Theta(n^2)$

we would need constants c1, c2>0 such that: $c1n^2 \leq 6n^3 \leq c2n^2$

Dividing by n^2: $c1 \leq 6n \leq c2$

But **grows without bound** as increases. Therefore, no constant can satisfy the upper bound.

Hence: $6n^3 \neq \Theta(n^2)$

Actually, $6n^3 = \Theta(n^3)$

because constant is ignored in asymptotic notation.

Q3(a) Briefly explain P and NP problems with suitable examples. [8 Marks]

P Problems - P (Polynomial time) is the class of problems that can be **solved in polynomial time** by a deterministic algorithm.

Examples:

- Binary Search → ($O(\log n)$)
- Merge Sort → ($O(n \log n)$)
- Shortest Path → polynomial time

Example: In Binary Search, an element can be found in ($O(\log n)$) time.

NP Problems - NP (Nondeterministic Polynomial time) is the class of problems for which a given solution can be **verified in polynomial time**.

In simple words: It may be difficult to find a solution, but if someone gives us a solution, we can check it efficiently.

Examples:

- 3-SAT
- Vertex Cover
- Hamiltonian Cycle
- Travelling Salesman Problem (decision version)

Example: In Vertex Cover, if someone gives us a set of vertices, we can check in polynomial time whether all edges are covered and whether the set has at most k vertices.

Relationship – P ⊆ NP

Every problem that can be solved in polynomial time can also have its solution verified in polynomial time.

Whether: P = NP is still an **open problem**.

Q3(b) If f(n)=O(g(n)), does it imply g(n)=O(f(n))? [5 Marks]

No, it does not always imply this.
f(n)=O(g(n)) means that g(n) is an **upper bound** on the growth of f(n).

Example

Let: $f(n) = n$ and $g(n)=n^2$

Then: $n = O(n^2)$ is true.

But: $n^2 = O(n)$ is **false**, because n^2 grows faster than n.

Therefore : $f(n) = O(g(n)) \not\Rightarrow g(n) = O(f(n))$

Only when both functions have the same asymptotic growth can we say: $f(n)=\Theta(g(n))$

Q3(c) "Best case analysis may not give clear idea of performance." [2 Marks]

The statement is **true**.

Best-case analysis considers only the input for which the algorithm performs **minimum operations**. Such an input may be rare and may not represent normal usage.

For example, Linear Search has best-case complexity: $O(1)$

when the required element is the first element, but normally the element may occur anywhere.

Therefore, **best-case analysis alone may not accurately represent the actual performance** of an algorithm.

Q4(a) What is SAT and 3-SAT? Prove 3-SAT is NP-Complete.

SAT (Boolean Satisfiability) is the problem of determining whether a given Boolean formula can be made **TRUE** by assigning TRUE/FALSE values to its variables.

Example: $F = (x \vee y) \wedge (\neg x \vee z)$

If some assignment makes (F) true, then the formula is **satisfiable**.

3-SAT is a special case of SAT problem where the Boolean formula is in **CNF (Conjunctive Normal Form)** and each clause contains **exactly 3 literals**.

Example: $(x1 \vee x2 \vee \neg x3) \wedge (\neg x1 \vee x3 \vee x4)$

The problem is to determine whether there exists an assignment that makes the complete formula TRUE.

Proof that 3-SAT is NP-Complete

To prove a problem NP-Complete, we show:

1. 3-SAT belongs to NP

If a truth assignment is given, we can check each clause in polynomial time.

Therefore: $3\text{-SAT} \in NP$

2. 3-SAT is NP-Hard

SAT is NP-Complete (Cook-Levin theorem).

Every SAT formula can be transformed into an equivalent 3-SAT formula in **polynomial time**.

Thus: SAT $\leq_p$ 3-SAT

Since SAT is NP-Complete, 3-SAT is NP-Hard.

Since 3-SAT is both **in NP**, and **NP-Hard**,

3-SAT is NP-Complete.

Q4(b) Best, Worst and Average Case Behaviour. Is average case the average of best and worst?

1. Best Case - The **best case** is the input for which the algorithm takes the **minimum number of operations**.

Example: Linear Search finds the element at the first position.

$T_{best}(n) = O(1)$

2. Worst Case - The **worst case** is the input for which the algorithm takes the **maximum number of operations**.

Example: Linear Search finds the element at the last position or does not find it.

$T_{worst}(n) = O(n)$

3. Average Case - The **average case** is the expected running time of an algorithm over **all possible inputs**, considering their probabilities.

For example, in Linear Search, if the element is equally likely to be at any position, the average number of comparisons is approximately: n + 1 / 2

so the average complexity is: O(n)

Is average-case efficiency the average of best and worst-case efficiencies?

No.

Average-case efficiency is **not** simply: Best + Worst / 2
It depends on:

- all possible inputs, and
- the probability/frequency of each input.

For example, if most inputs cause the algorithm to take near-worst-case time, the average will also be close to the worst case.

Average-case analysis is a probability-based expected performance over all inputs, not the arithmetic average of best-case and worst-case efficiencies.

Q3(b)2. Why is SAT important in Theoretical Computer Science?

SAT (Boolean Satisfiability Problem) is one of the most important problems in theoretical computer science because:

1. **First NP-Complete Problem:** SAT was the first problem proved to be **NP-Complete** by the Cook-Levin theorem.
2. **Basis for NP-Completeness:** Many other problems are proved NP-Complete by reducing SAT or its variants to them.
3. **Studies computational difficulty:** SAT helps us understand which problems can be solved efficiently and which are computationally difficult.
4. **Used in problem reductions:** It provides a standard way to compare the complexity of different problems.
5. **Practical applications:** SAT solving is used in areas such as **hardware verification, software testing, scheduling and planning**.
6.

Q4(b) Best, Average and Worst Case Analysis of Linear Search [7 Marks]

Case Analysis -

Best Case: Minimum number of operations for an input.

Average Case: Expected number of operations for all possible inputs.

Worst Case: Maximum number of operations for an input.

Given Algorithm: Linear Search

```
Linear-search(a, n, item) {
   for(i = 0; i < n; i++)    {
      if(a[i] == item)
         return a[i];
   }
   return -1;
}
```

1. Best Case

The item is found at the **first position**, i.e. i = 0.

Only **1 comparison** is required. $T_{best}(n)=O(1)$

2. Worst Case

The item is:

- at the **last position**, or
- **not present** in the array.

In both cases, all n elements are checked. $T_{worst}(n)=O(n)$

3. Average Case

Assuming the item is equally likely to occur at any position:

Comparisons required are: 1,2,3,…,n

Average comparisons: 1+2+⋯+n / n = n(n+1) / 2n = n + 1 / 2

Therefore: $T_{average}(n)=O(n)$

Case	Situation	Time
Best	Item at first position	$O(1)$
Average	Item at any random position	$O(n)$
Worst	Last position / not found	$O(n)$

Space Complexity: $O(1)$ because only a constant amount of extra memory is used.